

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 1 of 8

User uploads PLC file

Streamlit web interface — drag and drop your .st / .l5x / .xml file

```
# Streamlit upload widget
uploaded = st.file_uploader(
    "Upload PLC source file",
    type=["st", "l5x", "xml", "fbd"]
)
if uploaded:
    raw_code = uploaded.read().decode("utf-8")
```

formats: .st .l5x .xml .fbd max size: 2000 lines runs offline

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 2 of 8

Smart parser extracts structure
Python lxml + regex — no LLM yet, purely deterministic extraction

```
# Extract VAR blocks, I/O tags, timers
var_block = re.findall(
    r'VARs+(.*?)s+END_VAR', code, re.DOTALL
)
for line in var_block:
    name, dtype = line.split(':')
    if name.startswith(('DI_', 'DO_', 'AI_')):
        io_tags.append({name, dtype})
```

Variables found	24
I/O tags detected	11
Timer instances	3 (TON × 2, TOF × 1)
Function blocks	5

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 3 of 8

Zone classifier assigns each block
Rule-based keyword matching — deterministic, explainable, no hallucinations

<pre>zone_keywords = { "startup": ["INIT", "RESET", "READY"], "safety": ["ESTOP", "INTERLOCK"], "fault": ["FAULT", "ALARM", "TRIP"], "shutdown": ["STOP_CMD", "SAFE_STATE"], }</pre> # Unmatched lines → normal operation			
Startup sequence			18 lines
Normal operation			142 lines
Safety interlock			34 lines
Fault handling			28 lines
Shutdown sequence			22 lines



End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 4 of 8

Safety detector scans independently
Dedicated pass — ensures no safety tag is missed even inside normal operation code

```
estop = re.findall(  
    r'(E_STOP|ESTOP|EMERGENCY_STOP)s*:?=?s*(w+)',  
    code, re.I  
)  
risk = "HIGH" if estop else "LOW"
```

ESTOP tags found	DO_PumpStop, DO_ValveClose
Interlock conditions	3 detected
Risk level assigned	HIGH

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 5 of 8

I/O signal list auto-generated
A real ABB commissioning deliverable — built from parsed variable declarations

Tag name	Type · Zone
DI_PressureHigh	Digital In · Safety
DO_PumpStart	Digital Out · Normal
AI_FlowRate	Analog In · Normal
DO_AlarmBell	Digital Out · Fault
DI_GuardOpen	Digital In · Safety
+ 6 more tags...	11 total

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 6 of 8

LLM called 3× per zone
Claude API — engineer, auditor, technician prompts run separately per zone

```
# Called once per role per zone
response = client.messages.create(
    model="claude-sonnet-4-6",
    system=ROLE_PROMPTS["auditor"],
    messages=[{"role": "user",
                "content": zone_code["safety"]}]]
)
```

Zones processed	5
LLM calls made	15 (5 zones × 3 roles)
Tokens per call	~400 in, ~300 out
Time taken	~35 seconds total

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



Upload



Parse



Classify



Safety



I/O map



LLM



Diagram

8

PDF out

step 7 of 8

Control flow diagram generated

Graphviz renders a zone-transition visual — exported as PNG for the PDF

```
dot = Digraph(format='png')
dot.node('startup', 'Startup
(18 lines)')
dot.node('normal', 'Normal op
(142 lines)')
dot.node('safety', 'Safety
(34 lines)')
dot.edge('startup', 'normal')
dot.edge('normal', 'safety', label='trigger')
dot.render('flow', cleanup=True)
```

startup → normal → safety → shutdown fault path included

← Previous

7 / 8

Next →

End-to-end PLC processing pipeline — interactive step-by-step walkthrough



step 8 of 8

PDF assembled and ready to download
ReportLab builds the final report — all sections, tables, diagram, narratives

Section 1	Program overview + I/O signal list
Section 2	Engineer narrative (per zone)
Section 3	Auditor narrative (IEC 61511)
Section 4	Technician guide (plain steps)
Section 5	Safety summary + flow diagram
Total time	~55 seconds end-to-end
File size	~90 KB

[View sample output report ↗](#)